



SVEUČILIŠTE U ZAGREBU



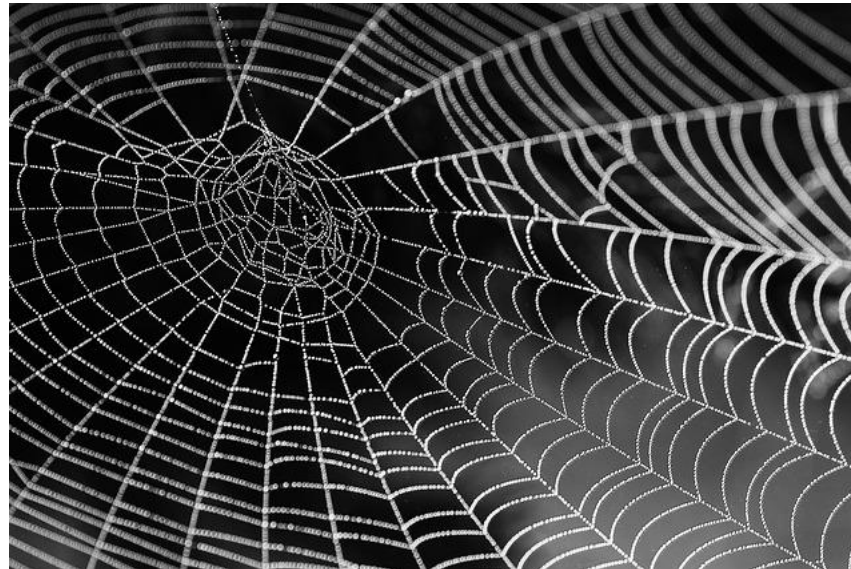
Fakultet
elektrotehnike i
računarstva

Prijediplomski studij

Računarstvo

Ak. god. 2025./2026.

Komunikacijske mreže 2



6. Protokol HTTP



Protokol HyperText Transfer Protocol (HTTP)

Hypertext Transfer Protocol

- koristi TCP, podrazumijevan *port* 80
- izvorno za prijenos hiperteksta, danas ima gotovo univerzalnu primjenu
 - prijenos datoteka (umjesto FTP-a)
 - webizirane usluge poput *webmaila*
 - *web services*
- razvoj
 - 0.9 - podržava prijenos samo hipertekstualnih dokumenata
 - HTTP/1.0
 - HTTP/1.1
 - HTTP/2
 - HTTP/3

Svojstva protokola

- globalni URI
 - razlika: URI, URL, URN (RFC 2396)
 - apsolutni i relativni URL
- razmjena zahtjeva i odgovora
 - uvijek klijent inicira komunikaciju
- ne čuva stanje - *stateless*
 - lakša izvedba, skalabilnost
- uključuje informacije o resursima
 - *metadata* - odnose se na resurs, ali nisu dio resursa
 - npr. duljina resursa, datum zadnje promjene, *encoding*

HTTP/1.0

- prvi standard - HTTP/1.0 - RFC 1945, svibanj 1996.
 - 2 draft-verzije 1993. godine
 - podržava prijenos različitih tipova podataka
 - posuđuje koncepte iz standarda MIME
 - zadržava kompatibilnost unazad
 - sredinom 90-tih - HTTP promet dominira - HTTP/1.0 neučinkovit
 - svako sjedište mora biti na drugom poslužitelju
 - neučinkovito korištenje TCP protokola (prevladavale male datoteke)
 - nema garancije za primanje dinamičkog sadržaja
 - nema podrške za upravljanje performansama - cache, proxy, djelomični dohvat, poštivanje preferenci klijenta
 - nema podrške za nastavak prekinutog prijenosa resursa
 - proizvođači preglednika i poslužitelja uvode svoja proširenja

HTTP/1.1

- verzija 1.1 definirana s RFC-om 2616 u lipnju 1999.
 - RFC 2617 - autentifikacija i sigurnost
 - **lipanj 2014: RFC 7230-7235**
- poboljšanja:
 - hop-by-hop, transfer coding
 - jedno fizičko računalo - više web poslužitelja - "virtualni host"
 - trajne konekcije - više zahtjeva preko jedne TCP konekcije
 - djelomični dohvat sadržaja
 - bolja podrška za cache i proxy – semantička transparentnost
 - pregovaranje o sadržaju datoteke (*content negotiation*)
 - bolji sigurnosni mehanizmi - autentifikacija

HTTP/2

- standardizacija počinje 2012., a završava 2015.
- standardi RFC 7540 (HTTP/2) i RFC 7541 (HPACK)
- binarni prijenos podataka
- uvedeni koncepti:
 - struja podataka
 - dvosmjerna, unutar konekcije
 - poruka
 - slijed okvira koji se smatra logičkom cjelinom
 - okvir
 - najmanja komunikacijska jedinica – ima zaglavlje koje identificira kojoj struji pripada

Protokoli QUIC i HTTP/3

QUIC ([RFC9000](#))

- Koristi UDP za prijenos
- Svojstva:
 - Malo kašnjenje kod uspostave konekcije
 - Multipleksiranje tokova
 - Autentifikacija i šifriranje
 - Kontrola zagušenja i oporavak od gubitaka
 - Upravljanje tokom i konekcijama
 - Migracija konekcije i otpornost na NAT *rebinding*
 - Pregovaranje o verzijama
- Google ga koristi na YouTubeu
 - Android app, Chrome (mobile/desktop)
 - Koristi se samo za prijenos video sadržaja

HTTP/3

- Dio funkcionalnosti iz HTTP/2 se prenosi na QUIC
- Svaki zahtjev/odgovor koristi zaseban QUIC tok
- Komprimiranje zaglavlja HPACK iz HTTP/2 je zamijenjeno QPACK-om ([RFC 9204](#))
- HTTP/3 koristi QUIC kao transportni sloj
- Predloženi standard [RFC 9114](#) (lipanj 2022.) – zrelo za korištenje
- [Podrška u preglednicima](#) 92%

HTTP 1.1

Primjer HTTP-zahtjeva

1. GET /predmet/rassus HTTP/1.1	početni redak
2. Host: www.fer.unizg.hr	polja zaglavlja: opća zaglavlja zaglavlja karakteristična za zahtjev ili odgovor zaglavlja karakteristična za entitet
3. User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.6; rv:7.0.1) Gecko/20100101 Firefox/7.0.1	
4. Accept: text/html,application/xhtml+xml,application/xml;q=0.8	
5. Accept-Language: hr,en;q=0.7,en-us;q=0.3	
6. Accept-Encoding: gzip, deflate	
7. Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.3	
8. Connection: keep-alive	
9. Cache-Control: max-age=0	
prazan redak	
tijelo poruke	

Primjer HTTP-zahtjeva

1. GET /predmet/rassus HTTP/1.1
2. Host: www.fer.unizg.hr
3. User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.6; rv:7.0.1) Gecko/20100101 Firefox/7.0.1
4. Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5. Accept-Language: hr,en;q=0.7,en-us;q=0.3
6. Accept-Encoding: gzip, deflate
7. Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
8. Connection: keep-alive
9. Cache-Control: max-age=0

Obavezni dijelovi

Primjer koristeći alat ncat (dio projekta nmap)

```
$ ncat -C www.fer.hr 80
```

```
GET / HTTP/1.1
```

```
Host: www.fer.hr
```

```
$ ncat -C --ssl www.fer.unizg.hr 443
```

```
GET / HTTP/1.1
```

```
Host: www.fer.unizg.hr
```

```
$ echo "GET / HTTP/1.1\r\nHost: www.fer.unizg.hr\r\n\r\n"  
| ncat --ssl www.fer.unizg.hr 443
```

Dokumentacija: <https://nmap.org/ncat/guide/index.html>

Metode zahtjeva

- metoda zahtjeva određuje što se traži od resursa
- HTTP/1.1 definira 8 metoda i omogućuje dodavanje novih metoda (*extensions*):
 - OPTIONS - informiranje i mogućnostima resursa i poslužitelja
 - **GET** - za dohvaćanje resursa (najčešća)
 - HEAD - za dohvaćanje podataka o resursu (npr. veličina, postojanje)
 - **POST** - aktiviranje resursa (npr. slanje podataka obrazaca)
 - PUT - postavljanje entiteta (npr. promjena podataka - kod REST-a)
 - DELETE - brisanje resursa (npr. kod REST-a)
 - TRACE - za dijagnostiku
 - CONNECT - za buduću uporabu (ne implementira se)
 - PATCH – parcijalno postavljanje entiteta (npr. promjena podataka - kod REST-a)

Primjer HTTP-odgovora

1.	HTTP/1.1 200 OK	početni redak
2.	Date: Wed, 12 Oct 2011 08:19:32 GMT	polja zaglavlja: opća zaglavlja zaglavlja karakteristična za zahtjev ili odgovor zaglavlja karakteristična za entitet
3.	Server: Apache/2.2.20 (FreeBSD) mod_ssl/2.2.20 OpenSSL/0.9.8q mod_fcgid/2.3.6	
4.	Expires: Thu, 19 Nov 1981 08:52:00 GMT	
5.	Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0	
6.	Pragma: no-cache	
7.	P3P: CP="NOI CURa ADMa DEVa TAla PSAa PSDa IVAa IVDa HISa OTPa OUR BUS IND UNI COM NAV INT"	
8.	Set-Cookie: CMS=2p72ge55h...	
9.	Vary: Accept-Encoding	prazan redak
10.	Transfer-Encoding: chunked	
11.	Content-Type: text/html; charset=utf-8	
12.		tijelo poruke
13.	d9e7	
14.	<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" " http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd ">	
15.	<html xmlns=" http://www.w3.org/1999/xhtml ">	
16.		
17.	<head>	
18.	<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />	
19.	...	

HTTP-odgovor

- početni redak (protokol, statusni kod i opis)
- kategorije statusnih kodova:
 - **1xx** – Informativne - ni uspjeh, ni neuspjeh
 - **2xx** – Uspjeh - poslužitelj je primio, razumio i ispunio zahtjev
 - **3xx** – Preusmjerenje - potrebno poduzeti dodatne akcije
 - **4xx** – Greška na klijentu - zahtjev je neispravan
 - **5xx** – Greška na poslužitelju - zahtjev je ispravan, ali ga poslužitelj ne može ispuniti
- u tijelu odgovora se obično prenosi reprezentacija resursa (“entitet”) koju preglednik treba prikazati korisniku (npr. HTML)
- primjer nekih polja zaglavlja:
 - **Content-Type:** format entiteta
 - **Content-Length:** duljina entiteta u tijelu (u oktetima)

Uspješan odgovor (2xx)

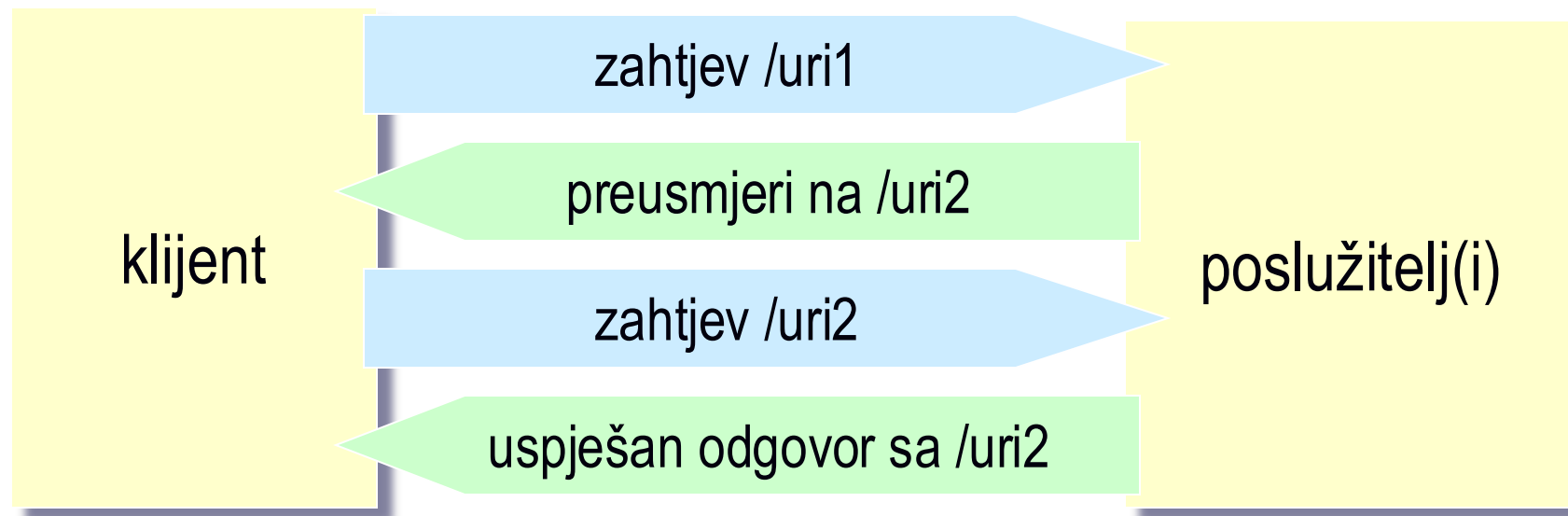
- poslužitelj je uspješno primio, razumio i ispunio zahtjev
- najčešći kôd **200** (*OK*)
 - za zahtjev **GET** znači da je dostavljeni entitet u tijelu odgovora sadržaj resursa
 - za **POST** znači da je resurs primio podatke i dostavljeni entitet opisuje ishod akcije
- najčešći odgovor na Webu

Drugi kôdovi uspješnog odgovora

- **201 (Created)** - rezultat metode PUT ili POST
- **204 (*No Content*)**
 - poslužitelj je ispunio zahtjev, ali nema potrebe da vraća ikakav entitet u tijelu odgovora (npr. anketa)
 - korisnički agent ne mijenja prikaz prošlog entiteta
- ...

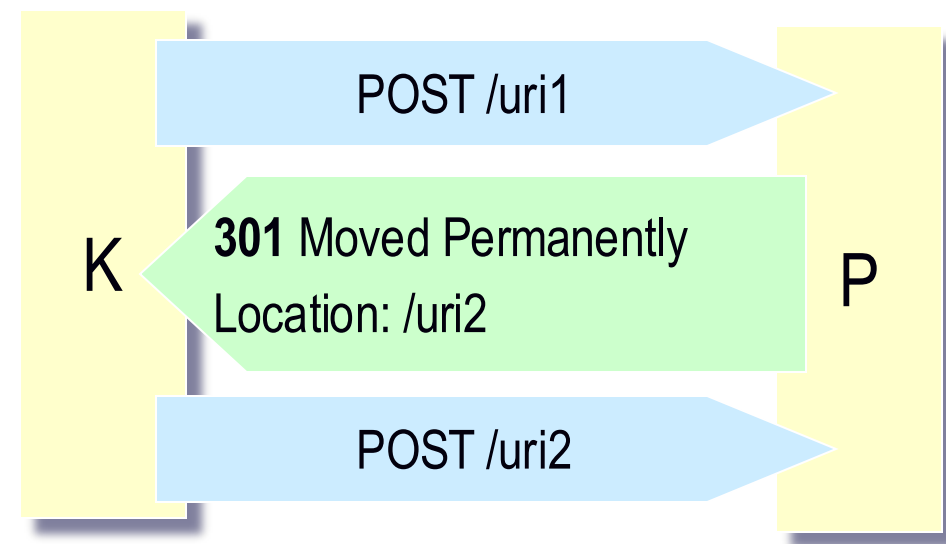
Preusmjerenje (3xx)

- klijent treba poduzeti dodatne korake kako bi ispunio izvorni zahtjev
- novi URI se nalazi u polju zaglavlja **Location**:
- sigurne metode se mogu izvršiti bez sudjelovanja korisnika
 - sigurne: GET, HEAD, OPTIONS, TRACE



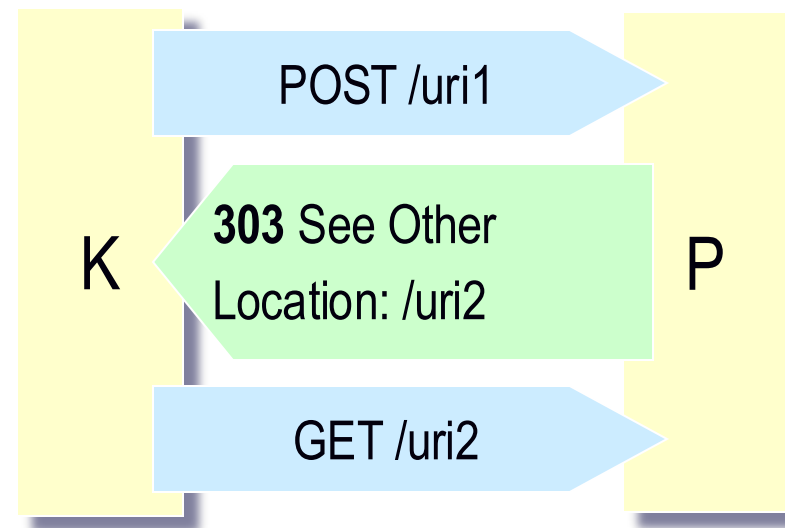
Kôdovi preusmjeravanja

- ponekad se resursu dodjeli drugi URI, pa se poslužitelj može konfigurirati da preusmjerava zahtjeve na novu adresu
- **301 (*Moved Permanently*)**
 - resurs je trajno premješten i svi bi budući zahtjevi trebali biti usmjereni na novi URI
- **302 (*Found*) - ranije Moved Temporarily**
 - resurs privremeno koristi drugi URI, klijent i dalje može koristiti stari URI
 - POST pretvara u GET (to je trebao raditi 303, browseri ne poštuju RFC)
- klijent ponavlja identičan zahtjev nad novim URI-jem
- ako metoda nije sigurna, traži se potvrda korisnika
- u tijelu odgovora se može nalaziti poruka za “ručno” preusmjeravanje



303 (See Other)

- zahtjev je ispunjen, ali sada treba prikazati sadržaj nekog drugog resursa (treba izvršiti **GET** nad resursom identificiranom u polju **Location**)
- budući da je GET siguran, preusmjeravanje se vrši automatski



Pogreške klijenta (4xx)

- namijenjena za slučajeve kad se čini da je pogreška nastupila na klijentovoj strani
 - razjašnjenja, status pregovora, duljina resursa, novi zahtjevi
- odgovor treba sadržavati poruku namijenjenu korisniku u kojem se opisuje situacija i nude rješenja
- **400 (*Bad Request*)**
 - pogreška u sintaksi zahtjeva
- **404 (*Not Found*)**
 - resurs nije dostupan, ali se ne ulazi u detalje zašto
 - obično je pogreška pri utipkavanju URI-ja
 - kreativne stranice ([poveznica](#))

Autorizacija (kôd 401)

- 401 (*Unauthorized*)
- zahtjevu nedostaje autorizacija korisnika

Više o tome na predmetu:
Sigurnost računalnih sustava

Pogreške poslužitelja (5xx)

- kod ovakvih odgovora poslužitelj je svjestan da zahtjev nije ispunjen zbog njegove pogreške
- **500** (*Internal Server Error*)
 - obično programska pogreška u resursu
- **503** (*Service Unavailable*)
 - poslužitelj je preopterećen
 - odgovor može sadržavati **Retry-After**: polje
- ...

Zaglavlja

- Podjela:
 - opća zaglavlja (*General headers*)
 - vrijede za obje strane (klijent i poslužitelj) i nisu vezana uz sadržaj poruke (npr. Cache-Control)
 - zaglavlja zahtjeva (*Request headers*)
 - šalje ih klijent, opisuju što je traženo ili preferencije korisnika (npr. Accept, User-Agent)
 - zaglavlja odgovora (*Response headers*)
 - šalje ih poslužitelj u odgovoru (npr. Content-Type, Location)
 - zaglavlja entiteta (*Entity headers*)
 - odnose se na poruku tijela, opisuju njegovu veličinu i karakteristike (npr. Content-Length, Content-Encoding).

Pregovaranje o sadržaju (*content negotiation*) (1)

- mehanizam kojim poslužitelj bira najprikladniji format sadržaja koji šalje klijentu
- klijent u zahtjevu šalje preferencije kroz zaglavlja:
 - Accept – navodi prihvatljive medijske tipove (npr. `text/html`, `application/json`)
 - Accept-Language – popis preferiranih jezika odgovora (npr. `en-US`, `hr-HR`)
 - Accept-Encoding – podržani algoritmi kompresije (`gzip`, `deflate`, `br`)
 - Accept-Charset – prioritetni skupovi znakova (`utf-8`, `iso-8859-2`)
 - Vary – pokazuje koja zaglavlja su utjecala na odabir formata
- poslužitelj odgovara što šalje natrag:
 - Content-Type – medijski tip koji šalje natrag
 - Content-Length – veličina sadržaja
 - Transfer-Encoding – ako ne znamo veličinu unaprijed

Pregovaranje o sadržaju (*content negotiation*) (2) zahtjev

GET / HTTP/1.1

Accept:

text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

Accept-Encoding: gzip, deflate, br, zstd

Accept-Language: hr-HR,en-US;q=0.9,en;q=0.8

Connection: keep-alive

Host: www.fer.unizg.hr

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)

AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36

...

Pregovaranje o sadržaju (*content negotiation*) (3) odgovor

HTTP/1.1 200 OK

Date: Sat, 08 Nov 2025 07:51:01 GMT

Server: Apache/2.4.41 (Ubuntu)

Content-Encoding: gzip

Vary: Accept, Accept-Encoding

Keep-Alive: timeout=5, max=100

Connection: Keep-Alive

Transfer-Encoding: chunked

Content-Type: text/html; charset=utf-8

...

Veličina tijela odgovora (1)

- Postoje dva načina:
 - zaglavlje Content-Length
 - kada znamo koliko je tijelo veliko
 - zaglavlje Transfer-Encoding: chunked
 - kada ne znamo koliko je veliko tijelo (kod dinamičkog sadržaja)
 - sadržaj se šalje u dijelovima (komadima)
 - format tijela:
 - **veličina komada u hex zapisu** — na početku svakog dijela se nalazi broj koji označava koliko okteta sadrži taj komad
 - **CRLF** — nakon veličine komada slijedi novi red (\r\n)
 - **podaci dijela** — slijedi točno onoliko okteta koliko je navedeno u veličini komada
 - **CRLF** — nakon podataka opet slijedi novi red (\r\n).
 - zadnji komad je veličine 0

Veličina tijela odgovora (2) – primjer

HTTP/1.1 200 OK

Transfer-Encoding: chunked

Content-Type: text/plain

4\r\n

Wiki\r\n

5\r\n

pedia\r\n

E\r\n

in\r\n\r\nchunks.\r\n

0\r\n

\r\n

1.komad

- veličina: 4
- sadržaj: "Wiki"

2.komad

- veličina: 5
- sadržaj: "pedia"

3.komad

- veličina: E → 14
- sadržaj: "in\r\n\r\nchunks."

4.komad

- veličina: 0
- to je kraj prijenosa

Privremeno spremanje i uvjetno dohvaćanje (1)

- privremeno spremanje (*cache*) na klijentu omogućuje lokalno spremanje kopije resursa radi bržeg pristupa i uštede propusnosti
- klijent koristi uvjetna zaglavlja (npr. `If-Modified-Since`, `If-None-Match`) pri ponovnim zahtjevima kako bi provjerio je li se resurs promijenio
 - ako nije, dobiva samo status `304 Not Modified` i koristi lokalnu kopiju, čime smanjuje promet
- privremeno spremanje može biti bilo gdje između klijenta i poslužitelja:
 - klijent, posrednik (*proxy*), poslužitelj

Privremeno spremanje i uvjetno dohvaćanje (2)

Zaglavlja:

- **Cache-Control** – upravljanje spremanjem resursa
- **If-Modified-Since** – šalje se da provjeri je li resurs promijenjen od određenog datuma
- **If-None-Match** – provjerava ETag za validaciju *cachea*
- **ETag** – jedinstveni identifikator verzije resursa (poslužitelj šalje klijentu)
- **Expires** – navodi kada resurs ističe i više nije valjan u spremniku
- **Last-Modified** – vrijeme zadnje promjene resursa

Privremeno spremanje i uvjetno dohvaćanje (3)

```
GET /_pub/themes_static/fer_2025/default/img/FERlogo.svg HTTP/1.1
```

Host: `www.fer.unizg.hr`

If-Modified-Since: Wed, 08 Oct 2025 12:25:48 GMT [sadržaj noviji od](#)

If-None-Match: "f974-640a4c8d30394" sadržaj zaglavlja je ETag

Cache-Control: max-age=0

traži najnoviju verziju

• • •

- klijent provjerava je li resurs promijenjen i traži da se spremljeni podatak koristi samo ako je aktualan prema zadnjoj promjeni u vremenu ili prema ETag vrijednosti (Etag ima prednost jer je precizniji)

Privremeno spremanje i uvjetno dohvaćanje (4)

HTTP/1.1 304 Not Modified

Date: Wed, 05 Nov 2025 10:25:00 GMT

ETag: "f974-640a4c8d30394"

ovo je jedinstveni identifikator

Cache-Control: max-age=3600

ova verzija se može spremiti na 3600 sekundi

...

- poslužitelj ne šalje tijelo
- klijent koristi lokalno spremljenu kopiju
- obično se ovo koristi za statičke sadržaje (slike, stilovi, skripte)
 - problem kod razvoja jer se skripte i stilovi skripte spremaju i preglednik niti ne šalje nove zahtjeve

Privremeno spremanje i uvjetno dohvaćanje (5)

HTTP/1.1 200 OK

ETag: "d41d8cd98f00b204e9800998ecf8427e" ovo je jedinstveni identifikator

Cache-Control: max-age=3600 ova verzija se može spremiti na 3600 sekundi

Last-Modified: Wed, 05 Nov 2025 09:00:00 GMT vrijeme zadnje promjene

Expires: Wed, 05 Nov 2025 11:00:00 GMT vrijeme kada ističe sadržaj

...

- kada imamo u odgovoru Cache-control i Expires
 - stari klijenti podržavaju samo Expires
 - ako klijent podržava Cache-control onda se to koristi

CORS (Cross-Origin Resource Sharing) (1)

- CORS je sigurnosni mehanizam temeljen na zaglavljima HTTP zahtjeva i odgovora koji dozvoljava klijentu da dohvaća resurse s drugih domena (*cross-origin*)
- preglednici primjenjuju pravilo istog izvora (*same-origin policy*) koje ograničava takve zahtjeve s drugih domena, sprječavajući neautorizirani pristup podacima između različitih domena
- za utvrđivanje domene se koristi zaglavlje `Origin` koji
- kod “*preflight*” zahtjeva klijent prvo provjerava dozvole prije slanja pravog zahtjeva

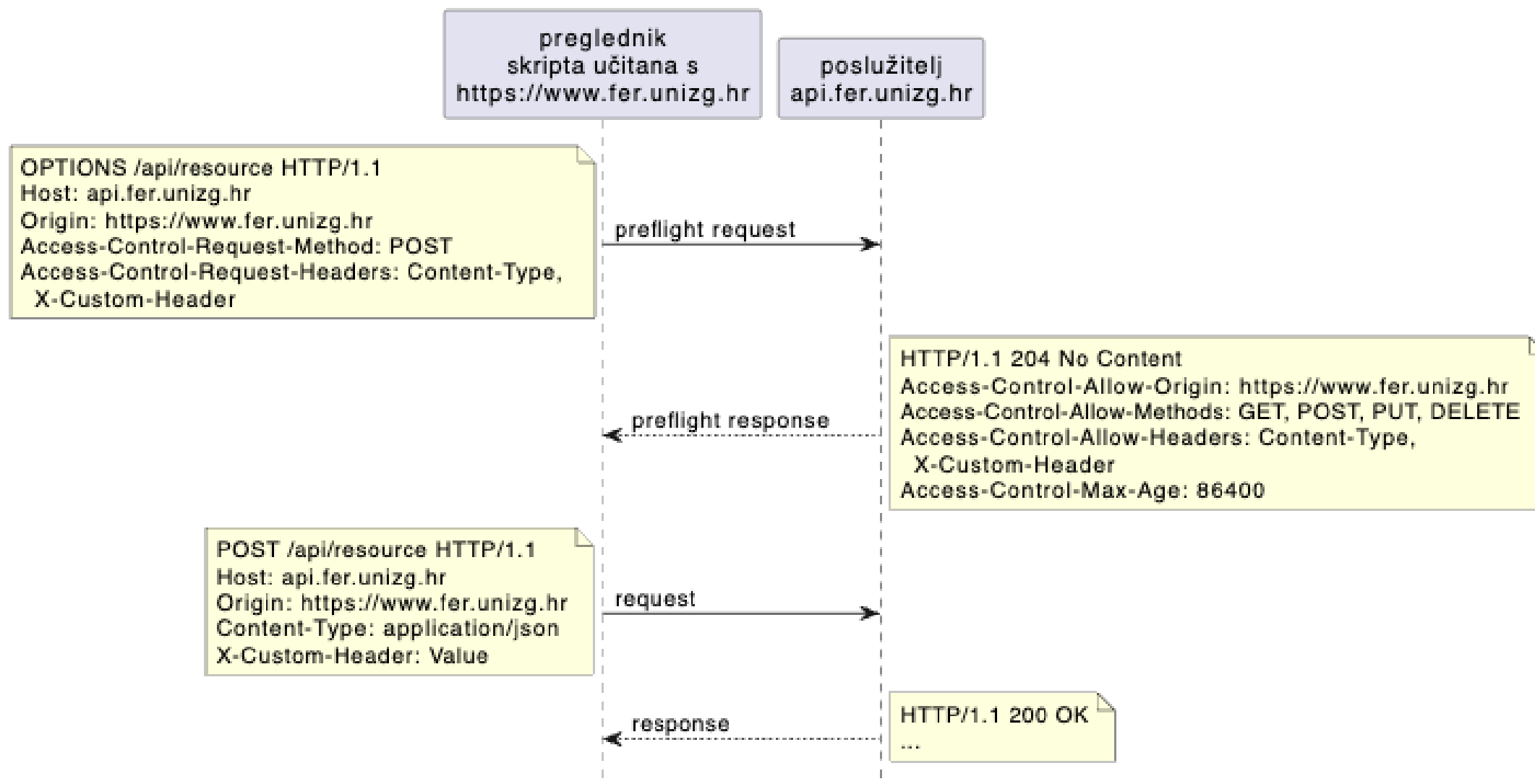
CORS (2) – zaglavlja

- **Access-Control-Allow-Origin** – određuje koje domene smiju pristupiti resursu
- **Access-Control-Allow-Methods** – navodi dozvoljene HTTP metode za *cross-origin* zahtjeve
- **Access-Control-Allow-Headers** – dopušta dodatna zaglavlja izvan standardnih
- **Access-Control-Expose-Headers** – specificira koja zaglavlja su dostupna klijentu kod
- **Access-Control-Max-Age** – koliko dugo *preflight* zahtjev može biti keširan.
- **Origin** – šalje klijent na početku svakog *cross-origin* zahtjeva

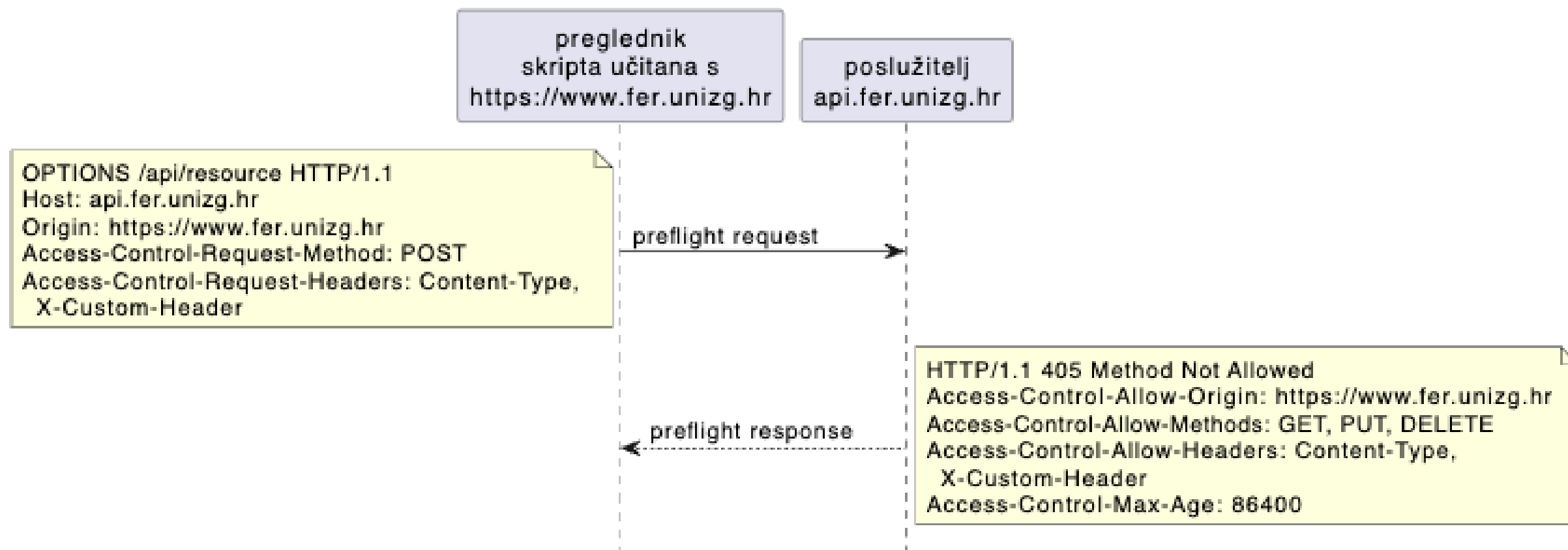
CORS (3) – jednostavni zahtjev

- preglednik šalje zahtjev ako je jednostavan (*simple request*) inače šalje prvo šalje *preflight* zahtjev pa tek onda pravi zahtjev
 - ako *preflight* ne uspije preglednik ne šalje pravi zahtjev
- jednostavni zahtjevi su:
 - koristi metode: GET, HEAD ili POST
 - POST može slati samo:
 - application/x-www-form-urlencoded,
 - multipart/form-data ili
 - text/plain

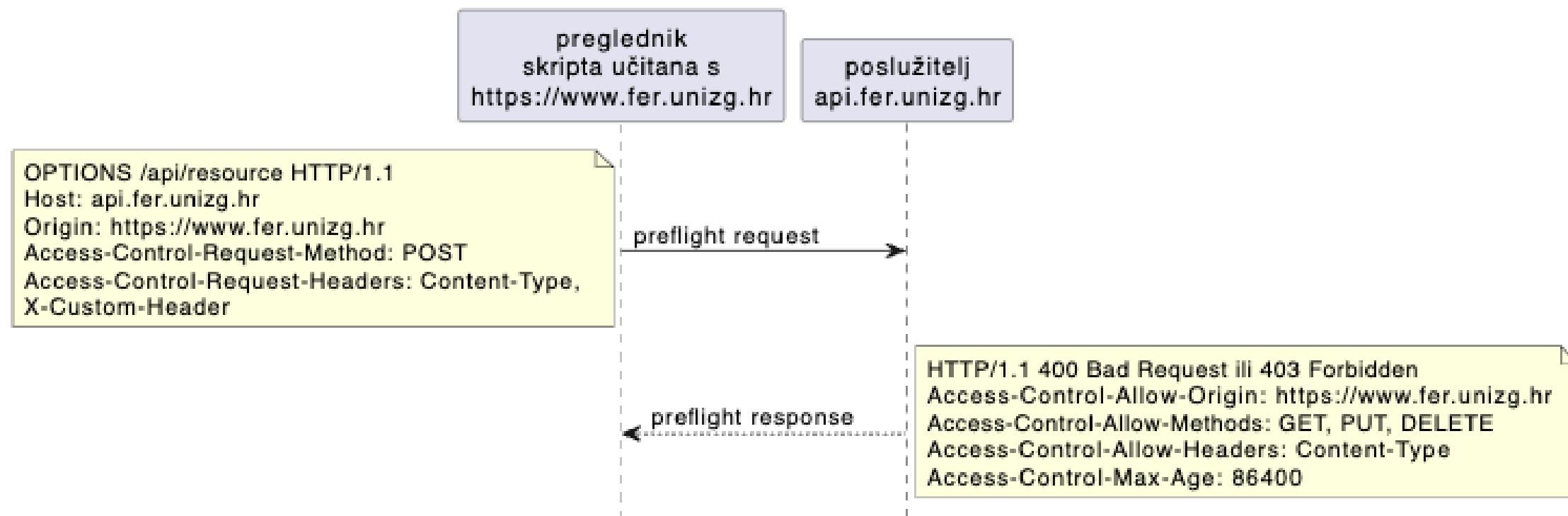
CORS (4) – uspješan poziv



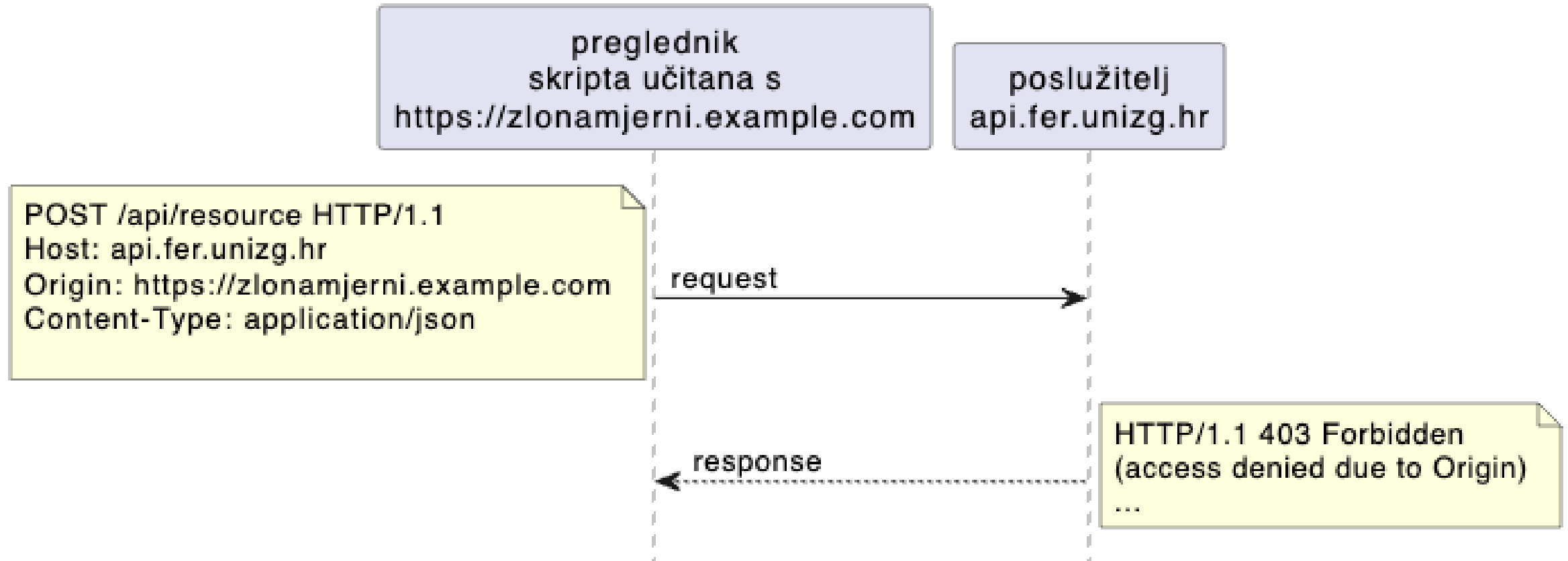
CORS (5) – neuspjeli *preflight* – metoda nije dozvoljena



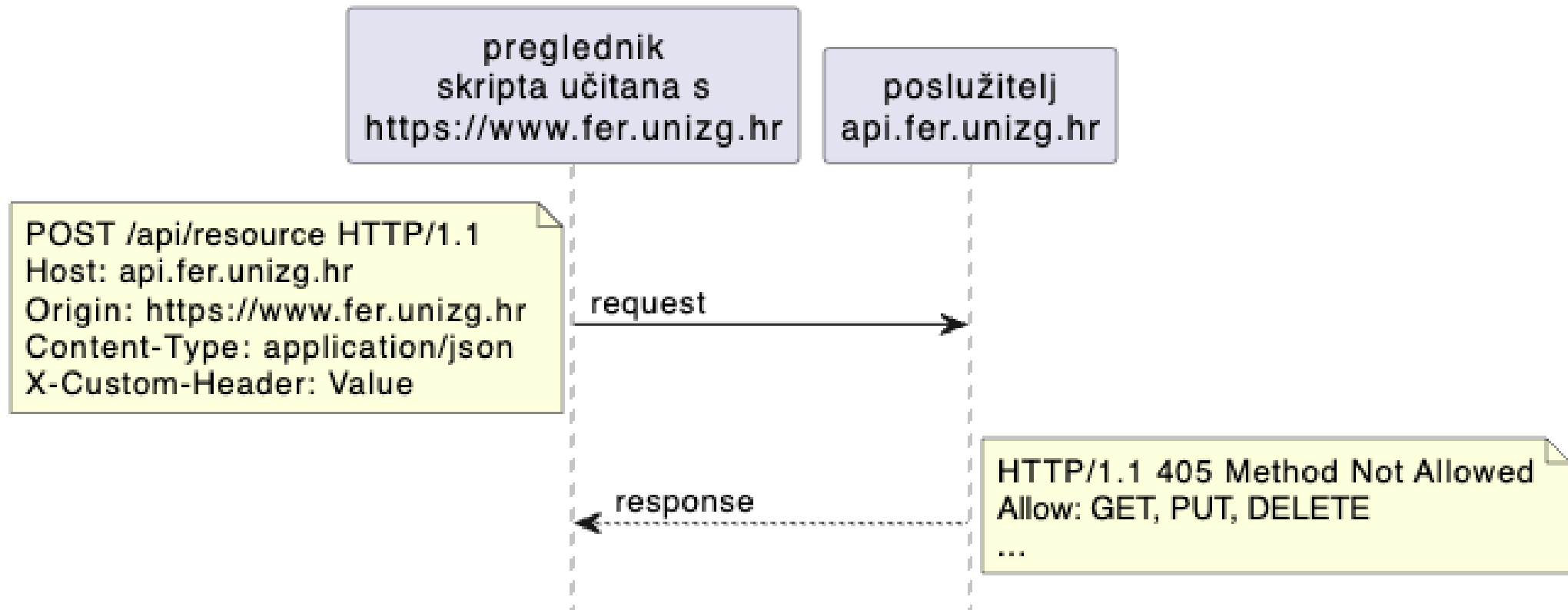
CORS (5) – neuspjeli *preflight* – zaglavlje nije dozvoljeno



CORS (6) – neuspjeli zahtjev zbog Origina

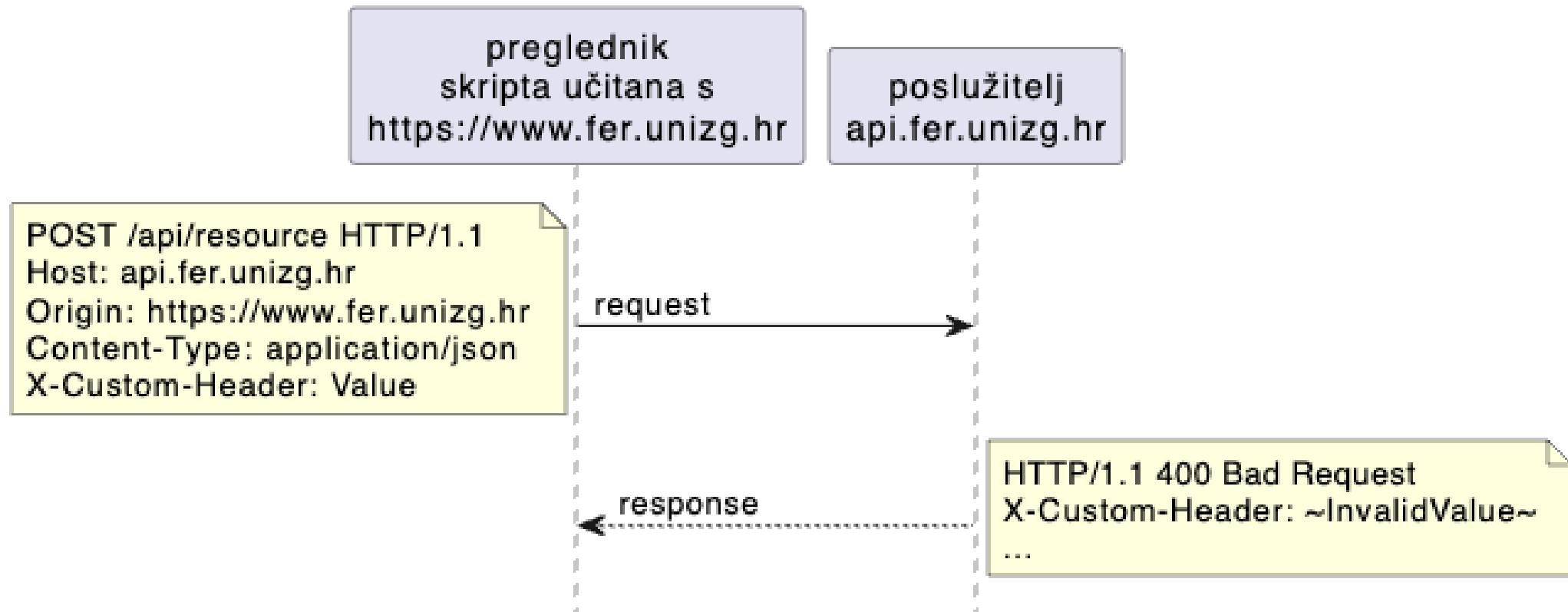


CORS (7) – neuspjeli zahtjev – nedozvoljena metoda



Ovo se ne bi trebalo dogoditi kada imamo preflight zahtjev

CORS (8) – neuspjeli zahtjev – nedozvoljeno zaglavlje



Ovo se ne bi trebalo dogoditi kada imamo preflight zahtjev